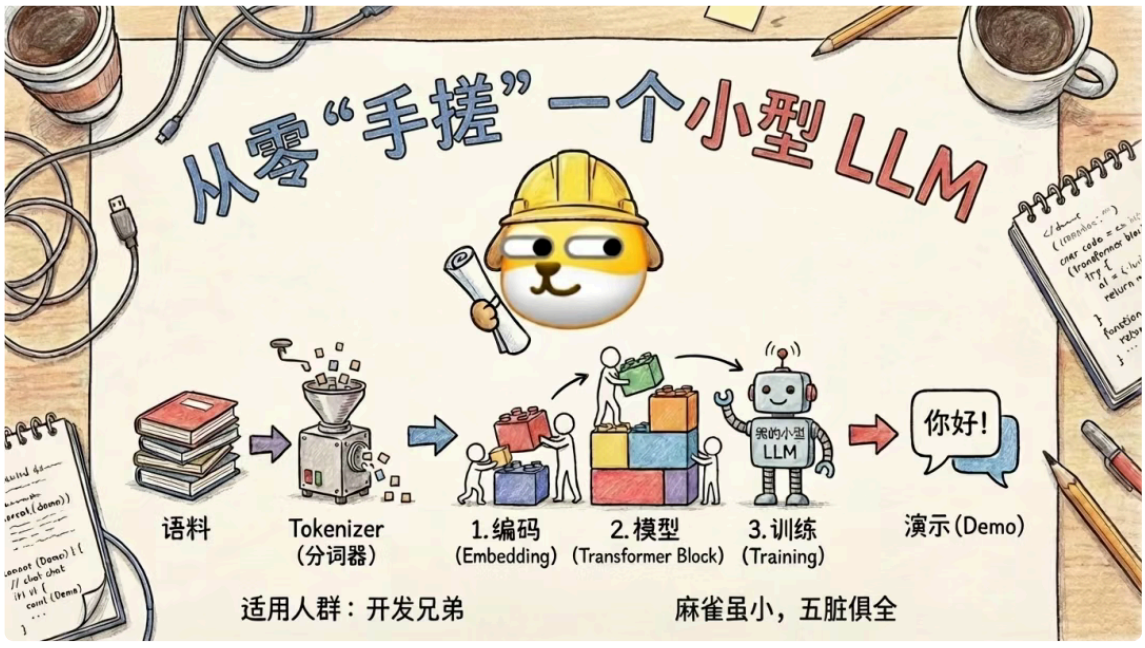
教你从零“手搓”一个大模型，别再只会调用API了

原创 腾讯程序员 腾讯技术工程 2025年12月24日 01:36 广东 2531人

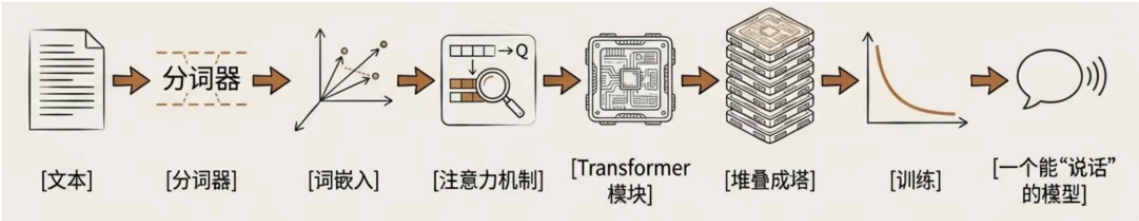


作者：yongzheng，patrickguo

> 在大模型满天飞的今天，我们是甘心做一个只会调用 OpenAI.chat.completions 的 API 搬运工，还是想真正弄懂那个黑盒子里到底发生了什么？本文将用最朴素的代码，完整的实现一个小型LLM。当你亲手写出 Attention 层的矩阵乘法，亲眼看到模型从输出乱码到学会说话，你会发现：大模型，其实也没那么可怕。



RoadMap



1. 引言

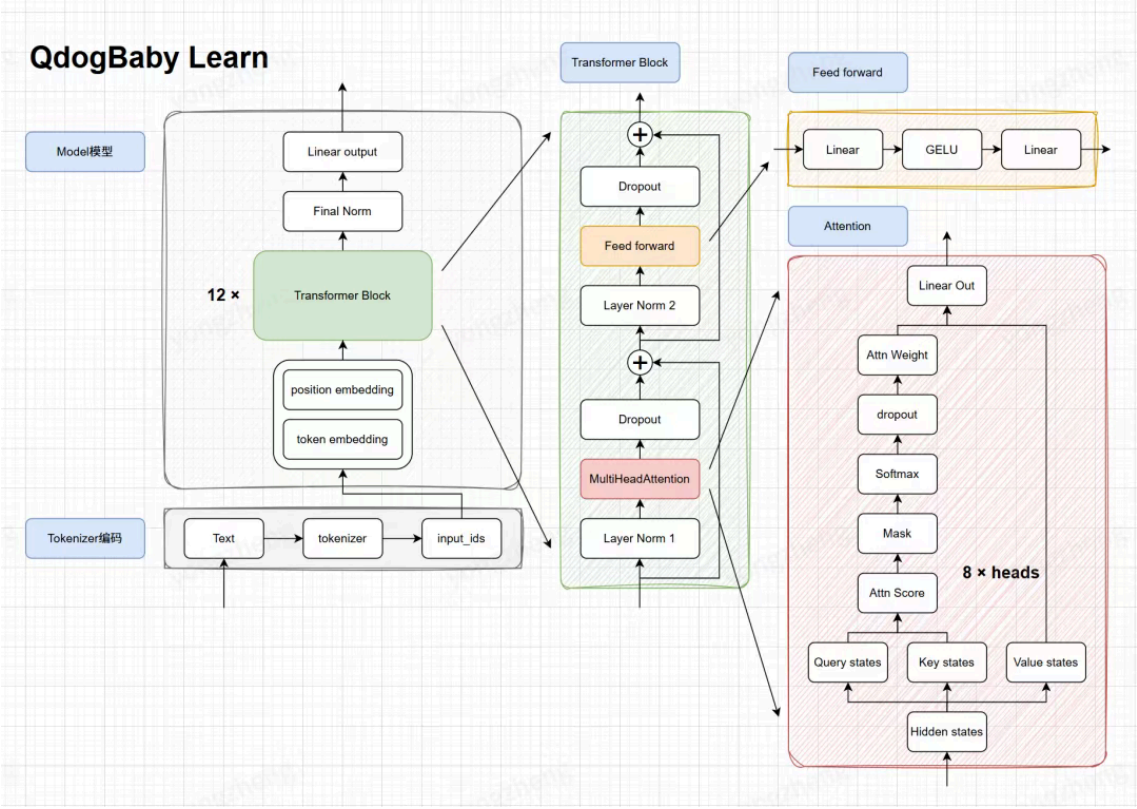
1.1 背景

在部门浓厚的AI技术探索氛围驱动下，为了将晦涩的理论转化为直观的工程触感，我们开展一次略带“整活”性质的实践。希望通过从零“手搓”的方式，在满足工程师好奇心的同时，帮助大家打破对大模型的神秘感，真正理解其底层的运行机制与实现原理。

1.2 内容说明

- 简单但完整的模型结构，结构主要参考GPT2，从零构建。
- 介绍模型结构以及代码实现，尽量规避数学与复杂的算法。
- 个人非科班的业余爱好者，靠各种碎片信息修炼的邪修，可能会对一些术语不了解，请见谅。
- 适应人群: 对于智商超群的算法同学来说可能有些幼稚，但对终日加班，目光涣散的开发苦哥们来说刚刚好。
- 主要参考资料: 《Build a Large Language Model (From Scratch)》 和 CS336: Language Modeling from Scratch 更多资料详见文末。

1.3 模型结构概览

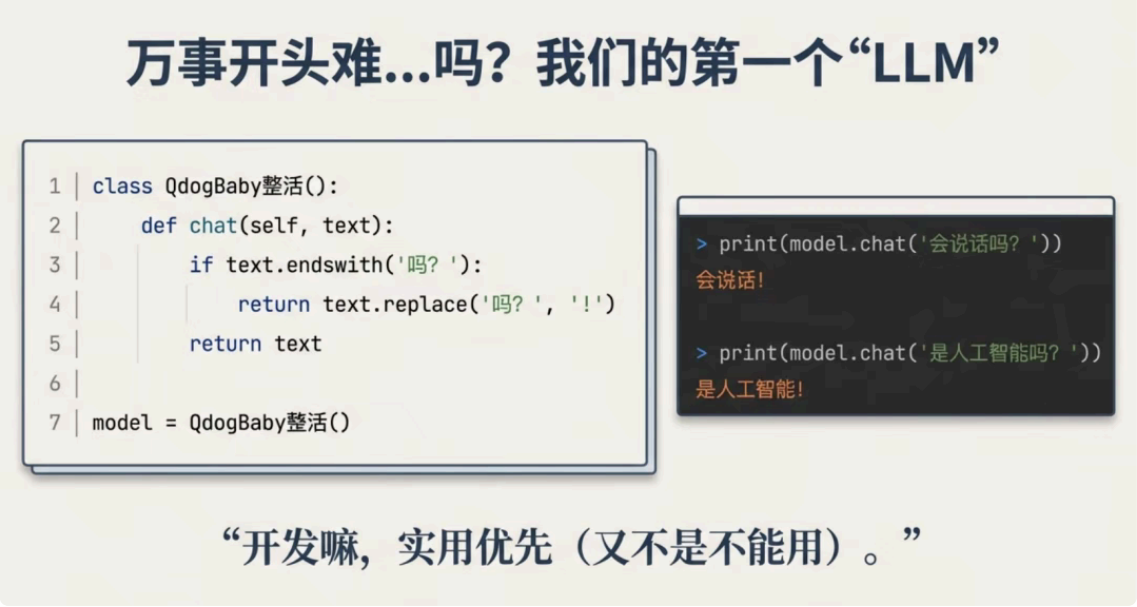


QDogBaby模型结构

现在看起来一头雾水没关系，接下来我们来逐步拆解模型，瞧瞧这些到底是怎么个事。

1.4 HelloWorld

首先，给我们的模型起个名字，然后做一个简单的逻辑，我们的第一个LLM就诞生了。开发嘛，实用优先(~~又不是不能用~~)




```
class QdogBaby整活():
    def chat(self, text):
        if text.endswith('吗？'):
            return text.replace('吗？', '！')
        return text
model = QdogBaby整活()
print(model.chat('会说话吗？'))
print(model.chat('是人工智能吗？'))
```

```
会说话！
是人工智能！ -->
```

这个“人工”智能最大的问题在于：它不懂语义，只是在做字符替换。它无法理解“今天天气不错”和“风和日丽”之间的联系，更无法生成它没见过的句子。

真正的 LLM 是一个概率模型。它不是在查找答案，而是在根据上文，预测下一个字最可能是什么。

为了实现从“逻辑处理”到“概率预测”的转变，我们首先要解决第一个问题：如何把人类的语言，翻译成模型能理解的语言？

2. 编码Tokenizer

2.1 引言

正如我们第一部分所说，LLM 是一个概率模型，通过数学计算来进行下一个字的预测。然而，计算机只认识数字（0和1），并不认识"今天"、"天气"或者"Today"这些单词。因此，想要让计算机能够理解人类的语言，需要通过特定的编码机制将自然语言翻译成模型能运算的数字

序列。分词器（Tokenizer）是实现这一功能的核心组件，它能够将连续文本拆解为离散语义单元（即Token）并转换为唯一索引（Token ID）用于模型输入。

所有 LLM 的输入都依赖 Tokenizer 的预处理结果。如果分词策略不合理，如拆分后丢失语义或 Token 粒度过细，会直接影响模型对文本的理解能力。因此，一个设计优良的 Tokenizer 是提升 LLM 性能的基础前提。

2.2 Tokenizer分类

根据切分粒度的不同，Tokenizer可分为三大类：字符级Tokenizer（Character-level）、词级Tokenizer（Word-level）以及子词级 Tokenizer（Subword-level）。我们以句子 "Today is sunday." 为例，看看它们是如何工作的。

字符级 Tokenizer（Character-level）

核心：以单个字符作为最小语义单元进行拆分，不依赖词典或复杂规则。

分词结果：`[T, o, d, a, y, i, s, s,u, n, d, a, y, .]`

优点：词汇表规模小，任何文本均能拆分，适配性强

缺点：Token序列长，信息密度低，碎片化严重，解码效率偏低

词级 Tokenizer（Word-level）

核心：以单个词语为最小语义单元进行拆分，通常使用空格或者标点符号进行分隔。

分词结果：`[Today, is, sunday, .]`

优点：单元语义完整，序列长度较短，信息密度高，模型无需学习字符组合关系。

缺点：词表规模庞大，训练成本升高。同时，若单词未出现在语料或词典中，会出现OOV(out-of-vocabulary)的现象。

子词级 Tokenizer（Subword-level）

核心：介于字符级与词级之间，常见的词保留完整，不常见的词拆分成有意义的片段（词根、词缀）。

分词结果：`[To, day, is, sun, day, .]`

优点：词汇表规模可控，序列长度介于两者之间，能学习词缀、词根之间的关联，平衡了信息密度与计算效率，同时有效缓解了OOV问题。

缺点：分词逻辑较上述两者复杂；对低频子词的拆分可能不够合理，需通过优化算法进行改善。

目前，子词级 Tokenizer 是 LLM 的主流选择。

2.3 BPE算法原理

目前最流行的子词分词算法是 BPE (Byte Pair Encoding)。它的核心逻辑与人类学习语言的过程相似：先认字，再组词。从最小的字符单元出发，反复合并语料中频率最高的相邻单元对，逐步扩展词汇表，直至达到预设词汇表大小或下一个高频出现的字符对频率为1时停止。

具体示例：

初始语料：`[apple, apple, banana, banana, grape, grape, grapes]`

步骤1：预处理，为原始语料中的每个单词添加后缀 `</w>` 作为分隔符，并将所有单词拆分为字符作为初始分词单元。

- `[a,p,p,l,e,</w>]` (2次)
- `[b,a,n,a,n,a,</w>]` (2次)
- `[g,r,a,p,e,</w>]` (2次)
- `[g,r,a,p,e,s,</w>]` (1次)

步骤2：初始化词汇表，此时词汇表仅包含独立字符与单词后缀。

```
V = {a, b, e, g, l, n, p, r, s, </w>}
```

步骤3：进行第一次合并。

步骤3.1：统计所有相邻单元对及其频率，需要叠加单词重复次数。

- `[a,p,p,l,e,</w>]` (2次)：相邻对为 `(a,p)`、`(p,p)`、`(p,l)`、`(l,e)`、`(e,</w>)` 各2次
- `[b,a,n,a,n,a,</w>]` (2次)：相邻对为 `(b,a)`、`(a,n)`、`(n,a)`、`(a,n)`、`(n,a)`、`(a,</w>)` 各2次
- `[g,r,a,p,e,</w>]` (2次)：相邻对为 `(g,r)`、`(r,a)`、`(a,p)`、`(p,e)`、`(e,</w>)` 各2次
- `[g,r,a,p,e,s,</w>]` (1次)：相邻对为 `(g,r)`、`(r,a)`、`(a,p)`、`(p,e)`、`(e,s)`、`(s,</w>)` 各1次

各单元对出现的频率依次为：5次 `(a,p)`、4次 `(a,n)`、4次 `(n,a)`、4次 `(e,</w>)` 等。

步骤3.2：选择频率最高的相邻对进行合并，当前 `(a,p)` 出现次数最多，因此把 `ap` 作为一个新元素加入词汇表，并进行词汇表的更新。

```
新词汇表：V = {ap, a, b, e, g, l, n, p, r, s, </w>}
```

步骤3.3：用合并后的词汇表重新进行单词的拆分。

- `[ap, p, l, e, </w>]` (2次)
- `[b, a, n, a, n, a, </w>]` (2次)

- [g, r, ap, e, </w>] (2次)
- [g, r, ap, e, s, </w>] (1次)

步骤4：依次类推，重新统计各元素出现的频率，重复步骤3进行合并，直至达到预设词汇表大小或下一个高频出现的字符对频率为1时停止。

最终分词逻辑：经过上述步骤得到词汇表后，按照Token长度进行排序，优先匹配最长的Token，如果匹配上则进行替换，直到单词的所有字符都被替换为词表中的Token为止。

2.4 编码

分词仅仅是第一步，计算机仍然不认识分词结果。我们需要将其转化为数字索引。在模型训练前，我们会生成一个巨大的映射表，给每一个 Token 分配一个唯一的ID。如：

Token	ID
...	...
apple	1023
to	1024
is	1025
...	...
grape	2045
day	2046
sun	2047
...	...
banana	8786
...	...

随后，我们可以对文本进行最后一步转换了。输入文本："today is sunday"，我们将其分为 ['to', 'day', 'is', 'sun', 'day'] ，查表映射，输出结果为 [1024, 2046, 1025, 2047, 2046] 。至此，我们解决了人机交流的第一道障碍，文本已经变成了整齐的数字序列。

接下来，我们需要把这些数字喂给模型，让它开始真正的"计算"。



腾讯技术工程

腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。

689篇原创内容

公众号

3. 模型 LLM Model

3.1 引导: 单字处理的窘境

将字转化为token_id之后，我们终于可以对文字做计算了，于是乎我搞了一个翻译模型，这个模型找到了汉字与单词之间的对应关系，会将给定的英文token，转化为中文token，比如:

```
model = QdogBabyTranslate()
print(model.translate("I like penguin"))
print(model.translate("We all like penguin"))
```

```
我喜欢小企鹅
我们都喜欢小企鹅
```

我觉得很开心，于是我拿去用，发现:

```
print(model.translate("How are you"))
print(model.translate("How old are you"))
```

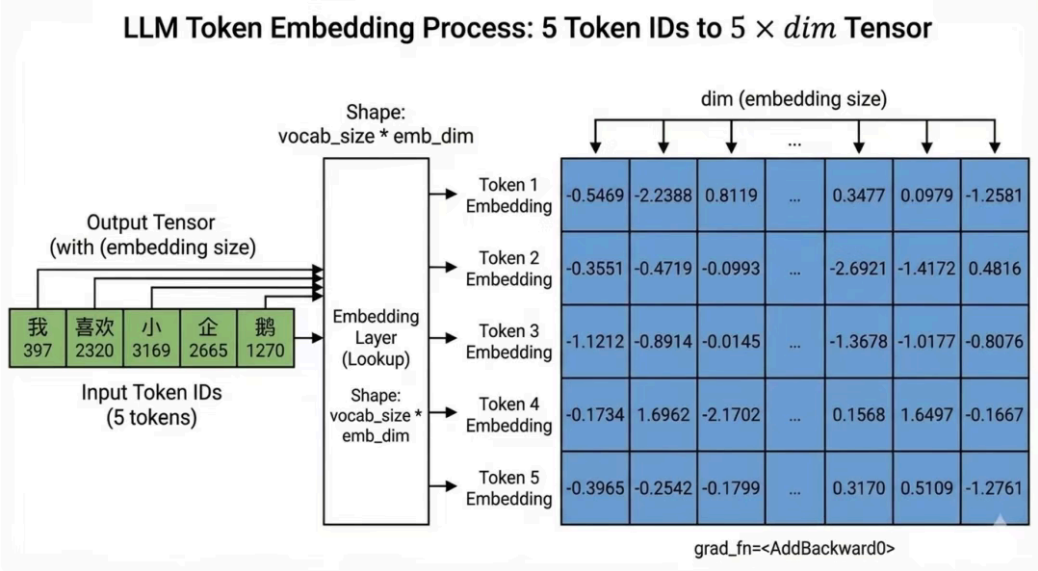
```
怎么是你
怎么老是你
```

感觉哪里怪怪的，它把 'old' 翻译成了 '老'，完全没看 'How' 和 'are' 的脸色！也就是说我们现在的每个字，**只关注了他自己本身，却忽略了对话上下文**，我们希望当前的文字可以**注意到**其他的文字。

3.2 embedding

在建立字与字的关系之前，还需要解决一个问题: 数字本身是没有语义的。我们将“我喜欢小企鹅”转换成了 [397, 2320, 3169, 2665, 1270]。但对模型来说，只是五个整数，它并不知道 397（我）和 2320（喜欢）之间有什么关系。如果我们直接用这些整数去计算，模型会“懵圈”。所以，我们要给每个 Token ID 做一次升维，把一个干巴巴的整数，变成一个高维向量。

首先需要准备一个巨大的表格，比如词表大小是 6400，维度是 512，那就是一个 6400 x 512 的数字矩阵，然后我们就可以通过最简单的查表的方式，将 Token ID 映射成一个高维向量。



此外，还有一个关键点：Transformer 是“脸盲”的。它并行处理所有输入，如果不加特殊处理，它无法区分“我喜欢小企鹅”和“小企鹅喜欢我”的区别（词都一样，只是顺序不同）。因此，我们还需要一个位置映射表（Positional Embedding），把“位置信息”也加到向量里去。

```python
from transformers import AutoTokenizer
import torch

tokenizer = AutoTokenizer.from_pretrained("./tokenizer")
input = tokenizer.encode("我喜欢小企鹅", return_tensors='pt')
print(input)

tok_emb = torch.nn.Embedding(num_embeddings=tokenizer.vocab_size, embedding_dim=512)
# 这里由于字在不同的位置，含义是不一样的，所以需要一个位置嵌入，来区分不同的位置
pos_emb = torch.nn.Embedding(num_embeddings=512, embedding_dim=512)
print(tok_emb(input)+pos_emb(torch.arange(input.shape[1])))
```

```
tensor([[ 397, 2320, 3169, 2665, 1270]])
tensor([[[-0.5469, -2.2388,  0.8119,  ...,  0.3477,  0.0979, -1.2581],
         [-0.3531, -0.4719, -0.0993,  ..., -2.6921, -1.4172,  0.4816],
         [-1.1212, -0.8914, -0.0145,  ..., -1.3678, -1.0177, -0.8076],
         [-0.1734,  1.6962, -2.1702,  ...,  0.1568,  1.6497, -0.1667],
         [-0.3965, -0.2542, -0.1799,  ...,  0.3170,  0.5109, -1.2761]]],
       grad_fn=<AddBackward0>)
```

你可以将这个操作理解为，将token在空间中展开，这样我们能更明显的感知到token之间的关系

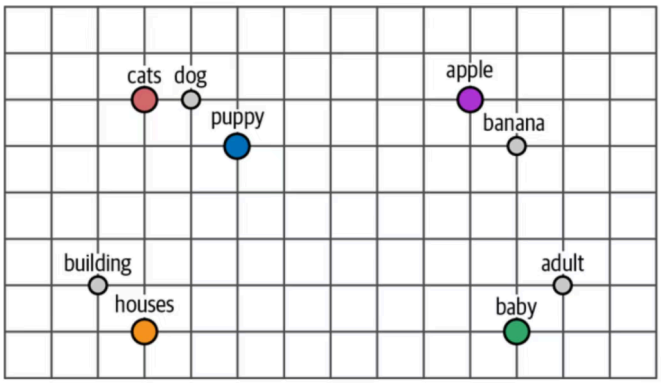


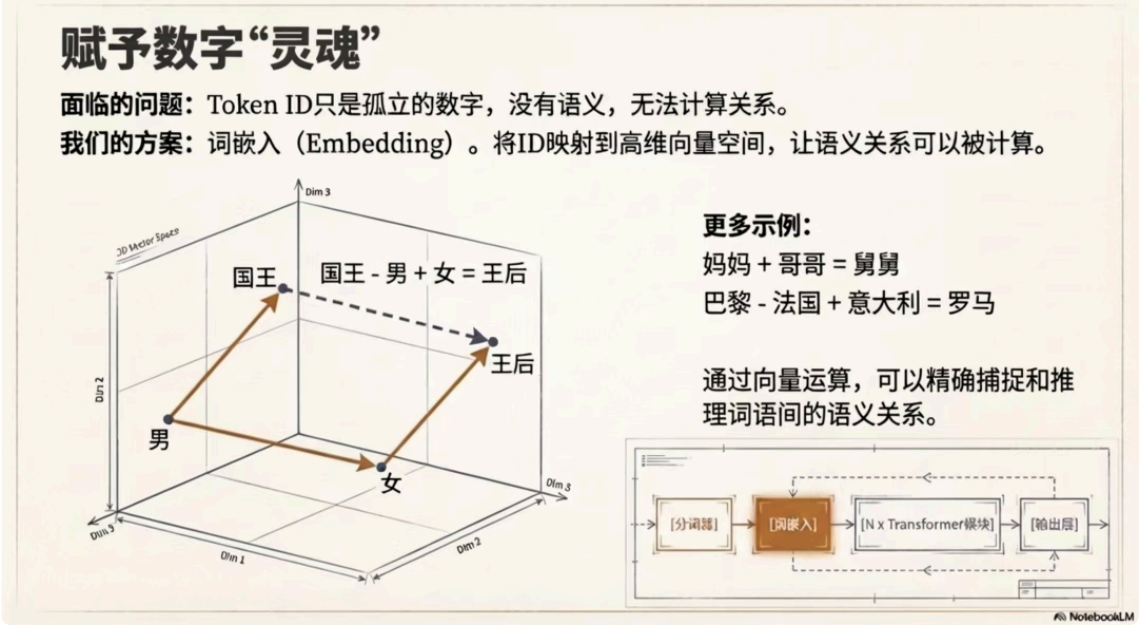
Figure 1-9. Embeddings of words that are similar will be close to each other in dimensional space.

另外，在token向量化后，我们发现可以形象的对token进行计算了，某种意义上可以完成这样的操作:

妈妈+哥哥=舅舅

国王-男+女=王后

巴黎-法国+意大利=罗马

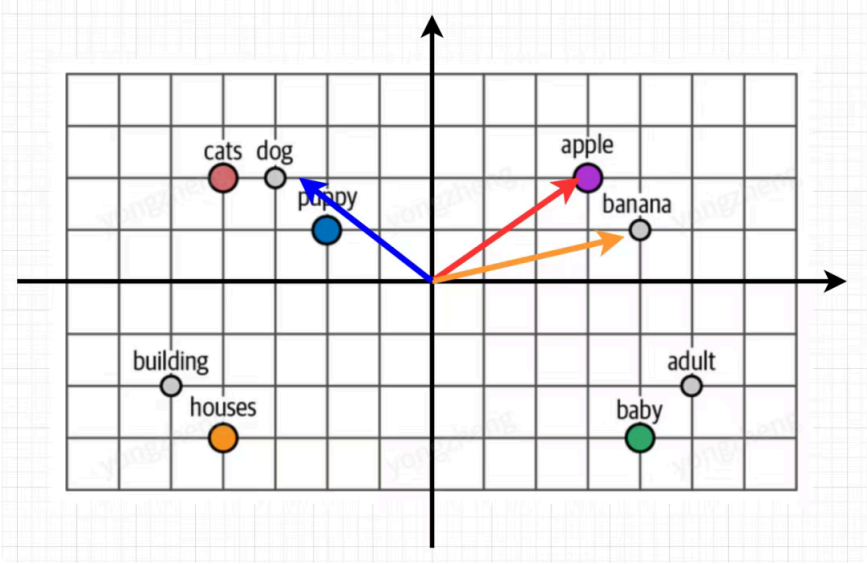


3.3 Attention

coding start: 这部分会涉及到一些编码，可能会比较繁琐无聊，但我认为还是有必要的，其他部分代码均有删减，但这部分保留了。

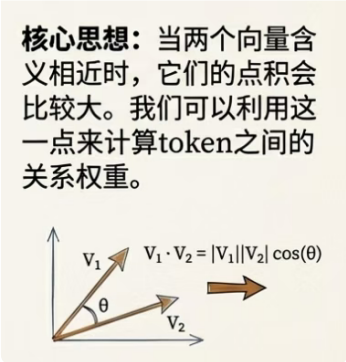
3.3.1 朴素的Attention

如何找到token与token之间的关系？回到我们刚刚embedding后的空间，如果我们在空间中建立一个坐标系，会发现，当两个向量含义相近时，他们的点积会比较大，而两个向量没什么关系时，它们的点积会是0甚至负数.

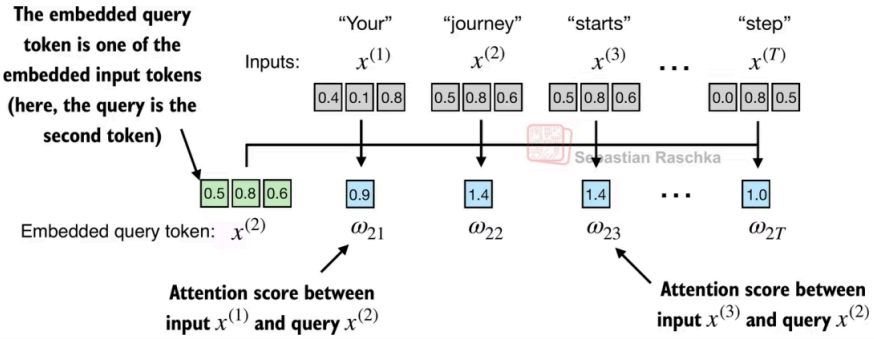


我们可以利用这个特性，将句子中的每个 Token 与其他所有 Token 进行点积运算(请注意这里的token是一个多维度的向量)，以此来计算这两个词的关系"好不好"。

向量点积的本质是空间投影，但对被需求淹死了的开发哥们来说存在理解难度，因此我们拿出初中知识，简单理解为离得近了(方向)点积大，离得远了(方向)点积小



图中为序列 "Your journey starts with one step"中第二个token"journey"的计算过程 (这里使用了原书中的例子，感兴趣的同学可以去阅读一下原书，讲解的十分详细)




```
inputs = torch.tensor(
  [[0.43, 0.15, 0.89], # Your     (x^1) - 第一个token的嵌入向量
   [0.55, 0.87, 0.66], # journey  (x^2) - 第二个token的嵌入向量
   [0.57, 0.85, 0.64], # starts   (x^3) - 第三个token的嵌入向量
   [0.22, 0.58, 0.33], # with     (x^4) - 第四个token的嵌入向量
   [0.77, 0.25, 0.10], # one      (x^5) - 第五个token的嵌入向量
   [0.05, 0.80, 0.55]] # step     (x^6) - 第六个token的嵌入向量
```

```
)
dim = inputs.shape[1]

attn_scores = torch.empty(6, 6)

# 计算注意力分数矩阵：遍历所有token对，计算它们之间的点积相似度
# 外层循环：遍历每个token作为查询(query)
for i, x_i in enumerate(inputs):
    # 内层循环：遍历每个token作为键(key)
    for j, x_j in enumerate(inputs):
        # 注意这里的token是一个多维度的向量，所以要展开计算
        dot_product = 0.0
        # 遍历向量的每个维度进行计算
        for k in range(len(x_i)):
            dot_product += x_i[k] * x_j[k]

        attn_scores[i, j] = dot_product
```

上面那一坨坨代码，可以写成下面这种矩阵相乘的形式，一行搞定。除了代码简化外，矩阵的计算还可以充分利用显卡的并行计算能力

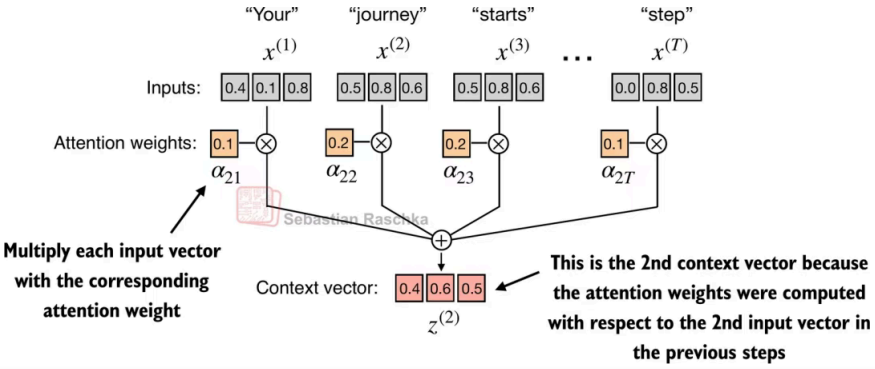
(感兴趣的小伙伴可以改一下这两段代码，测一下性能差距)

```
# inputs 形状: [seq_len, dim]
# inputs.T 形状: [dim, seq_len]
# 结果 attn_scores 形状: [seq_len, seq_len] -> 得到了一个 N*N 的关系矩阵
attn_scores = inputs @ inputs.T # query @ key
attn_scores
```

```
    tensor([[0.9995, 0.9544, 0.9422, 0.4753, 0.4576, 0.6310],
        [0.9544, 1.4950, 1.4754, 0.8434, 0.7070, 1.0865],
        [0.9422, 1.4754, 1.4570, 0.8296, 0.7154, 1.0605],
        [0.4753, 0.8434, 0.8296, 0.4937, 0.3474, 0.6565],
        [0.4576, 0.7070, 0.7154, 0.3474, 0.6654, 0.2935],
        [0.6310, 1.0865, 1.0605, 0.6565, 0.2935, 0.9450]])
```

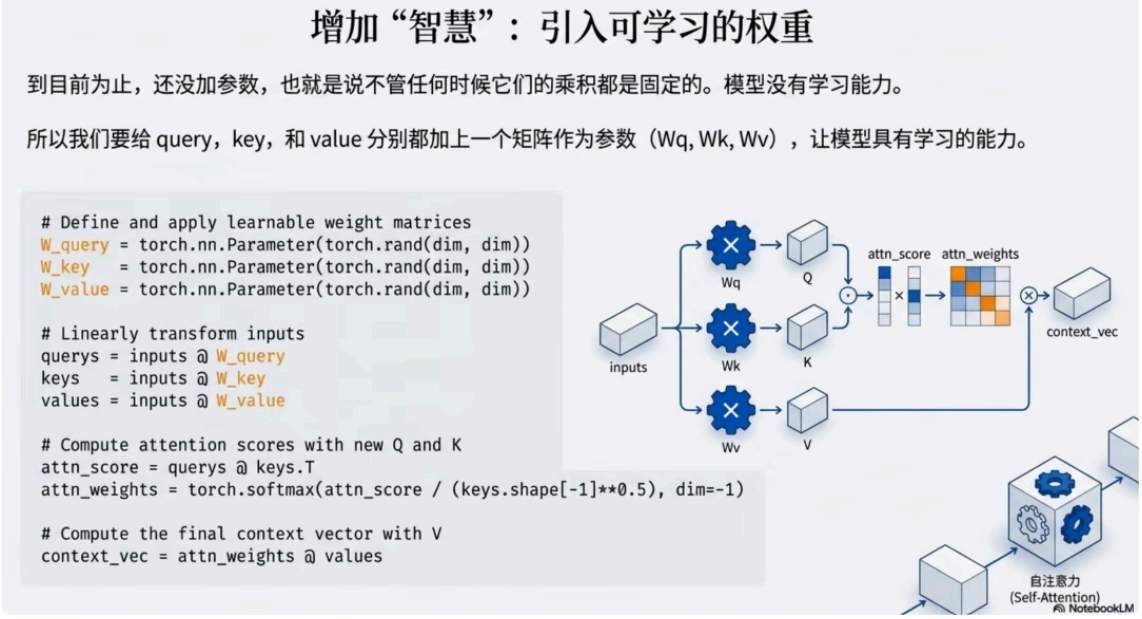
经过这样的计算后，我们得到了一个代表了词与词之间关系的矩阵，接下来还需要做两步操作: **Softmax 归一化** 和 **加权求和**。

由于原始的数据点积数值范围非常大，直接使用会使我们的计算结果不稳定，Softmax 的作用就是把这些乱七八糟的分数变成“百分比”(概率分布)，让它们加起来等于 1。这样我们就得到了每个词的权重，而加权求和则是真正的按这些权重，将我们需要的信息融合到一起。




```
# 1. Softmax: 将分数转化为概率（权重），相当于分配注意力比例
# 比如: [0.1, 0.8, 0.1] 表示主要关注中间那个词
attn_weight = torch.softmax(attn_scores, dim=-1)
# 2. MatMul: 根据权重，提取并融合信息
# 权重大的向量会被更多地“吸取”进来，权重小的则被忽略
context_vec = attn_weight @ inputs
```

3.3.2 增加权重



我们用一个小例子来通俗的理解一下attention计算的整个过程: 我们现在有一个会议室，然后来了几个token来开会，每个token在会议室门口根据会议室的规则(W_q, W_k, W_v)领到了三个角色:

Query: 提问者，当轮到他发言时，他会说出他需要的信息，比如"关于‘广告推荐架构’这个问题，谁有经验？"。

Key: 回答者, 当他被提问时，他会说出他对外的标签，比如"运维、后端、算法、美术"。

Value: 信息本身，如果他被选中了，他会将自己的信息贡献出来。

会议开始后，首先Query和Key关于当前的问题进行沟通，得到需要信息的比例，比如 "运维0.2、后端0.4、算法0.4、美术0"，对齐之后按照信息比例，把需要的Value获取到并融合起

来。会议结束时，每个人都通过这种方式，从其他人那里吸收了自己需要的信息，完成了信息的融合。

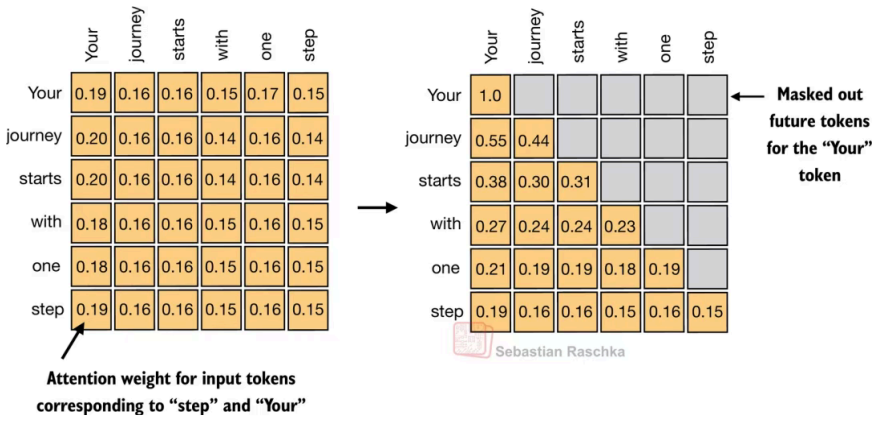
而用于讨论的会议室(W_q, W_k, W_v)，不是天然就存在的，需要我们使用我们训练的语料一点一点的盖起来。

```
dim = inputs.shape[1] # 词向量维度
# 定义三个可训练的参数矩阵
# 在实际 PyTorch 源码中，通常使用 nn.Linear 实现
W_query = torch.nn.Parameter(torch.rand(dim, dim))
W_key   = torch.nn.Parameter(torch.rand(dim, dim))
W_value = torch.nn.Parameter(torch.rand(dim, dim))
# 1. 线性变换：将输入映射为 Q, K, V
querys = inputs @ W_query
keys   = inputs @ W_key
values = inputs @ W_value
# 2. 计算注意力分数 (Scaled Dot-Product Attention)
# [seq_len, dim] @ [dim, seq_len] -> [seq_len, seq_len]
attn_scores = querys @ keys.T
# 3. 缩放 (Scaling) 与 归一化 (Softmax)
# 除以根号 dim，防止数值过大
scale_factor = keys.shape[-1] ** 0.5
attn_weights = torch.softmax(attn_scores / scale_factor, dim=-1)
# 4. 加权求和得到上下文向量
# [seq_len, seq_len] @ [seq_len, dim] -> [seq_len, dim]
context_vec = attn_weights @ values
```

coding end: 到这里就可以愉快的继续看图了

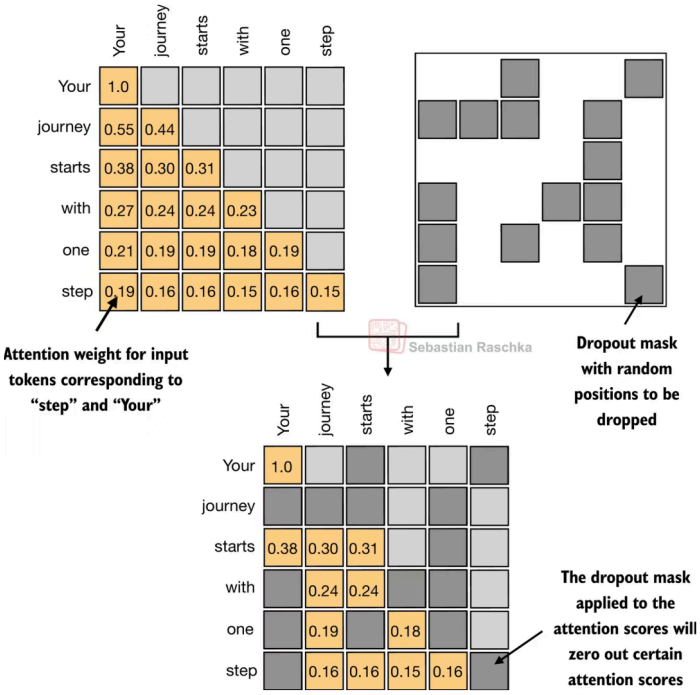
3.4 mask 和 dropout

我们的模型还存在着一个明显的问题，就是"看答案"。对于一些文本处理类任务，比如相关性计算、实体提取，会提前给定全部的语句，这种使用全部的token计算attention权重是没有问题的，但我们的模型是要"说话"的，在说出下一个字之前，没人知道后面要说什么。因此在计算权重时，我们要给当前词之后的词，都"遮起来"。

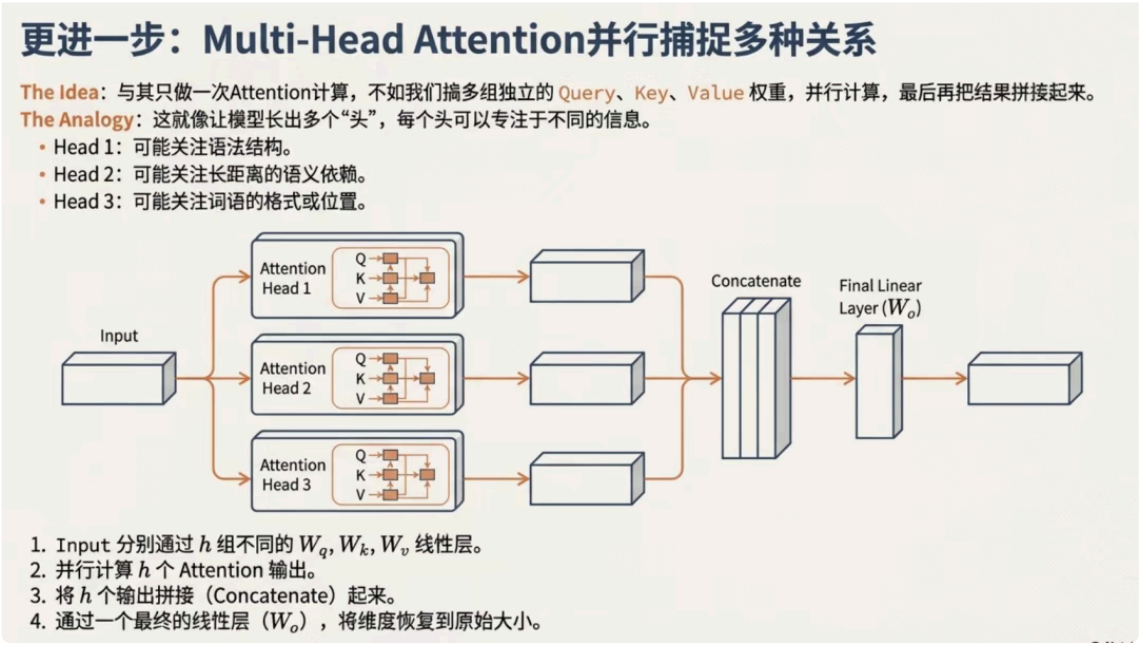


另外还有一种手段，会在计算过程中随机丢弃一些权重，可以防止模型过拟合，通俗的说就是我们希望模型可以学到知识，而不是背课文。

PS: 早期 GPT-2 和 Qwen-14B 采用了这个方式，但最新的 qwen3 kimi-2 等模型均将此参数设置为0



3.5 MultiHeadAttention

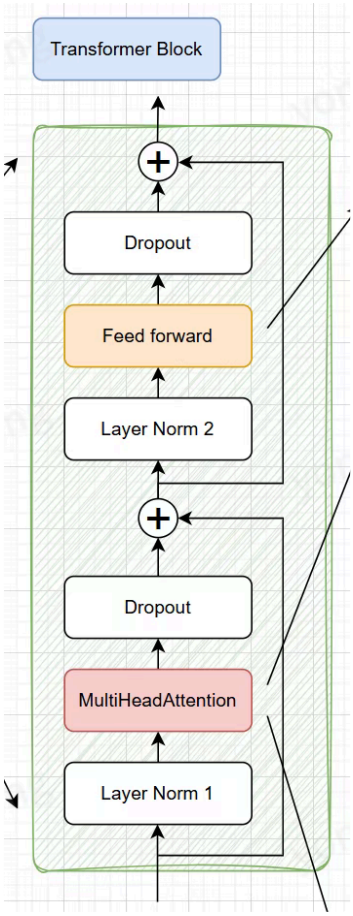


这里为什么最后要加一层linear将矩阵转成原来的形状？因为我们要开始叠层了~

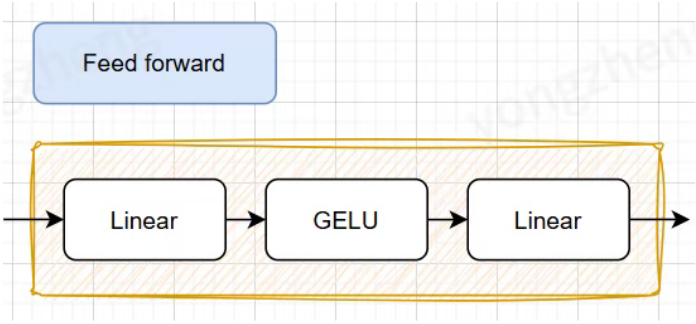
3.6 Transformer

我们参考2017年的著名论文《Attention Is All You Need》https://arxiv.org/abs/1706.03762

来设计我们的transformerblock，相对于论文原始结构，我们将Norm层提前，这是目前LLaMA、Qwen 等主流大模型普遍采用的结构，比原始 Transformer 的 Post-Norm 训练更稳定。



3.6.1 Feed Forward




```python
class FeedForward(nn.Module):
    def __init__(self, emb_dim, dropout):
        super().__init__()
        self.linear1 = nn.Linear(emb_dim, emb_dim * 4)
        self.linear2 = nn.Linear(emb_dim * 4, emb_dim)

    def forward(self, x):
        x = self.linear1(x)
        x = nn.GELU()(x)
        x = self.linear2(x)
        return x
```

这里的FeedForward主要负责两件事情:

1. 增加了非线性的激活函数 GELU()

关于激活函数，不做过多展开，可以先简单理解为不加非线性函数的话，多层数学上就等价于单层，层数就叠不起来。

2. 通过Linear将维度升为 4 * dim，假设我们的模型维度为512，那么这里的维度就是2048。(这里的4倍有说法的，感兴趣的同学可以研究一下)

这里的主要目的还是提高模型参数，增加模型的拟合能力。

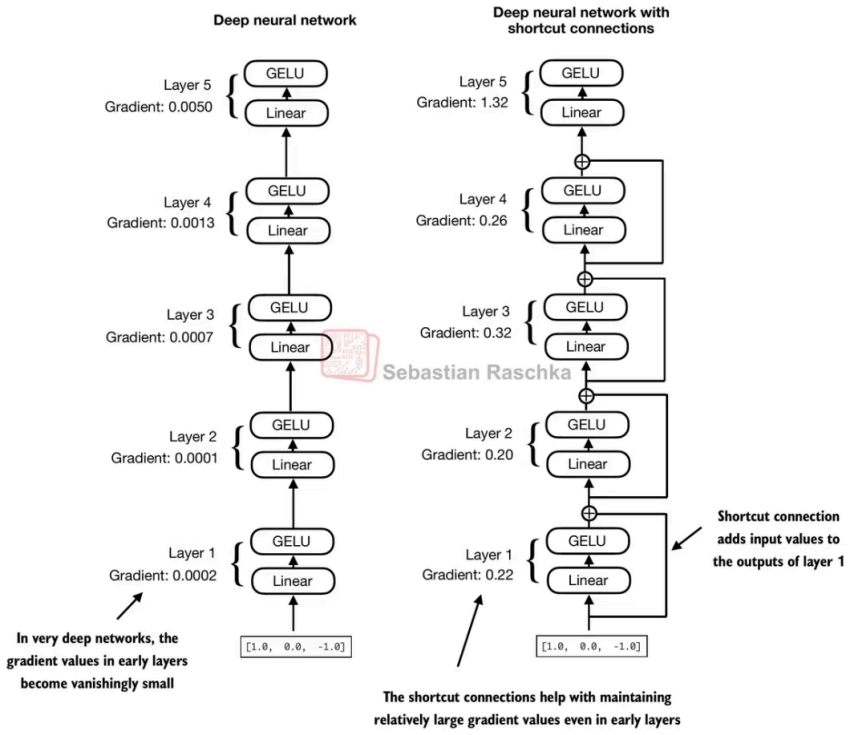
我们可以通俗的理解为: 我们前面加上参数的attention都是向量运算，而向量运算本质上是一个向量空间向另一个向量空间的转化，所以这个操作本质上可以认为是将词向量从原始的语义空间内拉扯到我们当前这句话的语义空间内，相当于让模型学了我们这里的方言。但学完了之后模型还需要"思考"(或者"搜索")。因此在这里给模型更多的参数，让他充分的理解这块的方言。

毕竟"学而不思则罔，思而不学则殆"嘛，说明孔子在2500年前已经领悟到LLM的真谛。

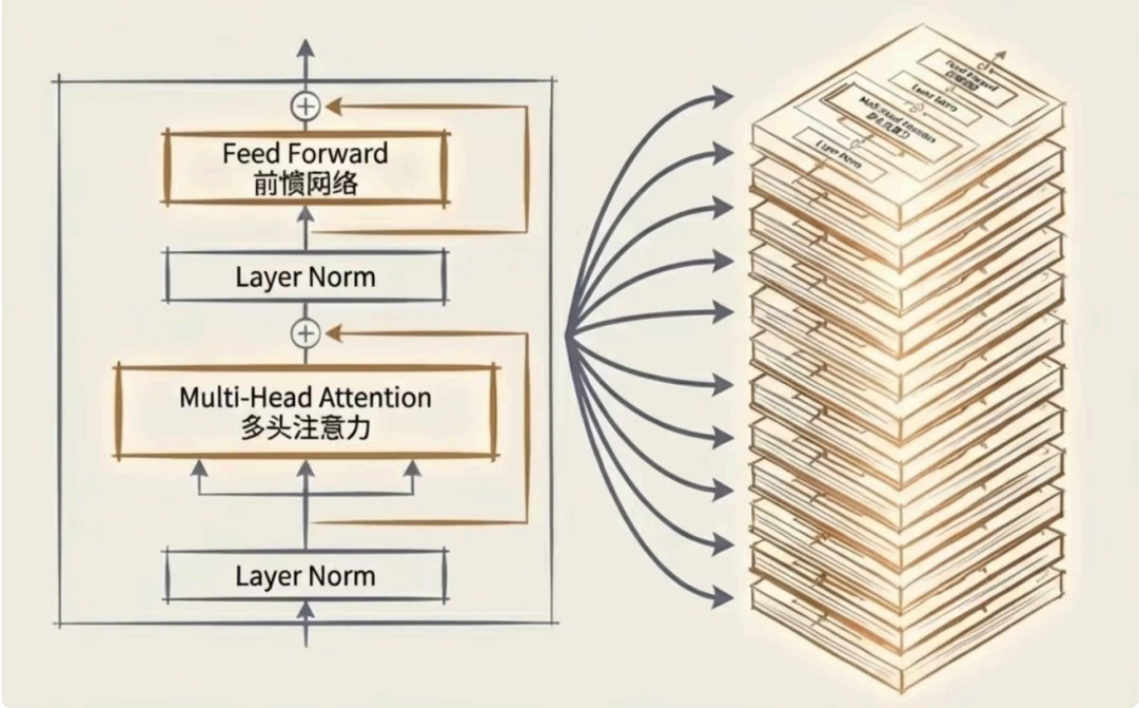
以上为邪修猜想，当个乐子看。

3.6.2 Residual

我们训练的主要过程就是求梯度、求导，如果无脑叠层，很容易就给梯度整没了，所以这里给原始的input和我们计算过后的output连起来，保证梯度不消失，毕竟"做模型嘛，不能忘本"。

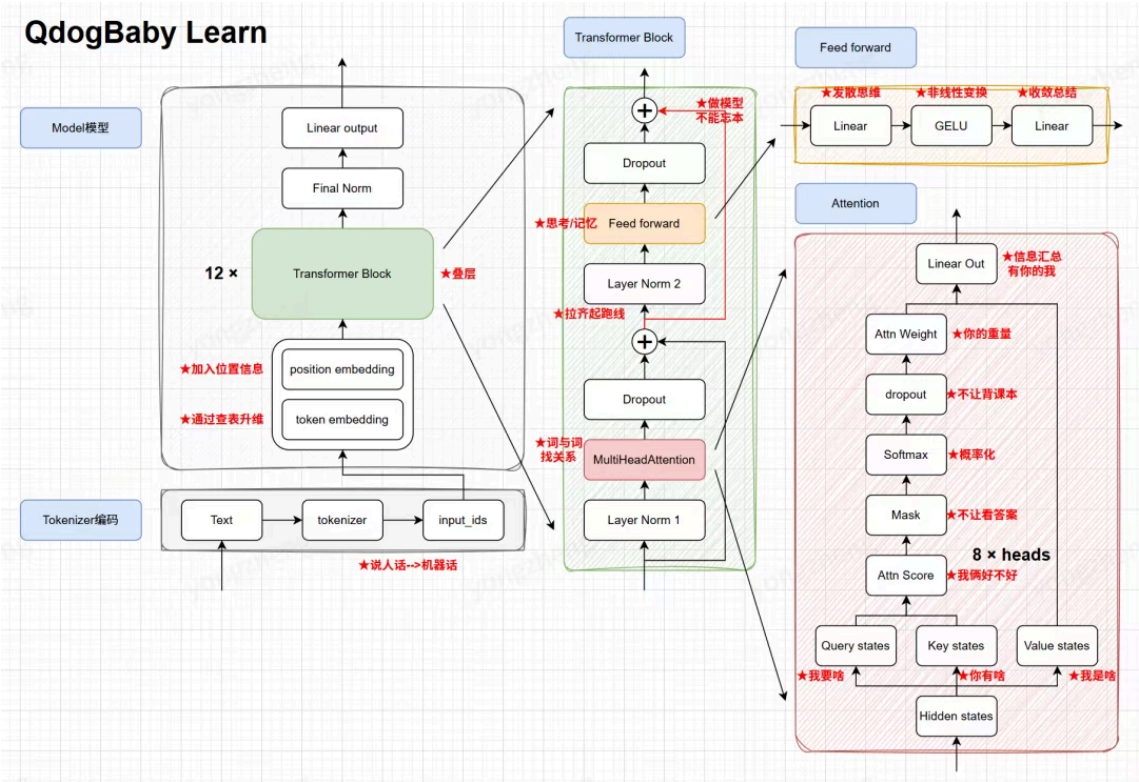


在解决了这些问题之后，我们的模型就可以开始叠层了



3.7 LLM模型

至此，我们主要的模型结构就完成了，我们结合标注来再次复习一下:



QDogBaby模型结构

```python
import torch
from torch import nn



# ========================================
# 1. 配置清单（相当于装修前的设计图纸）
```

```python
# ========================================
class QdogBabyLearnConfig():
    def __init__(self):
        self.model_name = "qdogbabylearn"
        self.version = "1.0.0"

        self.num_hidden_layers = 16# 盖多少层楼（Transformer Block的数量）
        self.num_heads = 8           # 雇多少个专家同时看（多头注意力的头数）
        self.emb_dim = 512           # 词向量的宽度（每个词包含多少信息量）
        self.dropout = 0.0           # 随机扔掉多少神经元（防止死记硬背）
        self.context_size = 512      # 一次能看多长的文章（上下文窗口大小）
        self.vocab_size = 6400       # 字典里有多少个字（词表大小）


# ========================================
# 2. 模型主体（QdogBabyLearn 机器人本体）
# ========================================
class QdogBabyLearnLLM(nn.Module):
    def __init__(self, config: QdogBabyLearnConfig):
        super().__init__()
        self.config = config

        # ★ 查表升维：把离散的字变成向量
        self.tok_emb = nn.Embedding(config.vocab_size, config.emb_dim)
        # ★ 加入位置信息：让模型知道“我爱你”和“你爱我”顺序不同
        self.pos_emb = nn.Embedding(config.context_size, config.emb_dim)

        self.dropout = nn.Dropout(config.dropout)

        # ★ 叠罗汉：堆叠 16 层 Transformer Block
        self.transformer_blocks = nn.Sequential(
            *[TransformerBlock(config.emb_dim, config.num_heads, config.dropout, config.c
              for _ in range(config.num_hidden_layers)]
        )

        # 最后输出前的标准化
        self.norm = LayerNorm(config.emb_dim)
        # ★ 映射回字典：把向量变回预测下一个字的概率分数
        self.out = nn.Linear(config.emb_dim, config.vocab_size)

    def forward(self, x):
        # x 的形状：[batch_size, seq_len]（比如：[[1, 2, 3]]）

        # 1. 查词表
        tok_embeds = self.tok_emb(x)
        # 2. 查位置表
        pos_embeds = self.pos_emb(torch.arange(x.shape[1], device=x.device))
```

```
        # 3. 信息融合（词义 + 位置）
        x = tok_embeds + pos_embeds
        x = self.dropout(x)

        # 4. 经过 16 层“思考”
        x = self.transformer_blocks(x)

        # 5. 最终整理
        x = self.norm(x)
        x = self.out(x) # 输出形状: [batch_size, seq_len, vocab_size]
        return x


# ========================================
# 3. Transformer Block（模型的大脑皮层）
# ========================================
class TransformerBlock(nn.Module):
    def __init__(self, emb_dim, num_heads, dropout, context_size):
        super().__init__()
        self.multi_head_attn = MultiHeadAttention(emb_dim, num_heads, dropout, context_si
        self.layer_norm1 = LayerNorm(emb_dim)
        self.feed_forward = FeedForward(emb_dim, dropout)
        self.layer_norm2 = LayerNorm(emb_dim)

    def forward(self, x):
        # --- 第一步：注意力机制（词与词找关系）---
        residual = x  # ★ 做模型不能忘本：先记住原始输入
        x = self.layer_norm1(x) # ★ 拉齐起跑线
        x = residual + self.multi_head_attn(x) # 原始信息 + 新学到的关系

        # --- 第二步：前馈网络（思考/记忆）---
        residual = x  # ★ 再次记住当前状态
        x = self.layer_norm2(x) # ★ 拉齐起跑线
        x = residual + self.feed_forward(x) # 原始信息 + 思考后的结果

        return x


# ========================================
# 4. 多头注意力（核心：我要啥、你有啥、我是啥）
# ========================================
class MultiHeadAttention(nn.Module):
    def __init__(self, emb_dim, num_heads, dropout, context_size):
        super().__init__()
        self.emb_dim = emb_dim
        self.num_heads = num_heads
        self.head_dim = emb_dim // num_heads # 每个头负责的维度（512/8 = 64）
```

```
        # 定义 Q, K, V 的映射层
        self.W_querys = nn.Linear(emb_dim, emb_dim) # 我要啥
        self.W_keys = nn.Linear(emb_dim, emb_dim)   # 你有啥
        self.W_values = nn.Linear(emb_dim, emb_dim) # 我是啥

        self.out = nn.Linear(emb_dim, emb_dim) # 信息汇总
        self.dropout = nn.Dropout(dropout)

        # ★ Mask (不让看答案): 创建一个下三角矩阵
        # 1 0 0
        # 1 1 0
        # 1 1 1
        self.register_buffer("mask", torch.tril(torch.ones(context_size, context_size)))

    def forward(self, x):
        batch_size, num_tokens, emb_dim = x.shape

        # 1. 线性变换生成 Q, K, V
        querys = self.W_querys(x)
        keys = self.W_keys(x)
        values = self.W_values(x)

        # 2. 切分成多头 (8个人分工合作)
        # 形状变换: [B, T, 512] -> [B, T, 8, 64]
        querys = querys.view(batch_size, num_tokens, self.num_heads, self.head_dim)
        keys = keys.view(batch_size, num_tokens, self.num_heads, self.head_dim)
        values = values.view(batch_size, num_tokens, self.num_heads, self.head_dim)

        # 3. 转置, 把头(Head)放到前面, 方便并行计算
        # 形状变换: [B, 8, T, 64]
        querys = querys.transpose(1, 2)
        keys = keys.transpose(1, 2)
        values = values.transpose(1, 2)

        # 4. 计算相似度 (我俩好不好)
        # Q @ K^T
        attn_score = querys @ keys.transpose(-1, -2)

        # 5. ★ Mask 操作 (不让偷看后面的词)
        # 把 mask 为 0 的位置填成负无穷大 (-inf)
        mask_bool = self.mask.bool()[:num_tokens, :num_tokens]
        attn_score = attn_score.masked_fill(mask_bool == 0, float('-inf'))

        # 6. 归一化 (概率化)
        # 除以根号dk是为了防止梯度消失
        attn_weight = torch.softmax(attn_score / (self.head_dim ** 0.5), dim=-1)
        attn_weight = self.dropout(attn_weight)
```

```
        # 7. 加权求和（有你的我）
        # Weight @ V
        context_vec = attn_weight @ values

        # 8. 还原形状（把8个头的结果拼回去）
        # [B, 8, T, 64] -> [B, T, 8, 64] -> [B, T, 512]
        context_vec = context_vec.transpose(1, 2).contiguous().view(batch_size, num_token

        # 9. 最终线性输出
        context_vec = self.out(context_vec)

        return context_vec


# ============================================
# 5. 前馈网络（发散思维 -> 收敛总结）
# ============================================
class FeedForward(nn.Module):
    def __init__(self, emb_dim, dropout):
        super().__init__()
        # ★ 发散思维：维度放大 4 倍（512 -> 2048）
        self.linear1 = nn.Linear(emb_dim, emb_dim * 4)
        # ★ 收敛总结：维度变回原样（2048 -> 512）
        self.linear2 = nn.Linear(emb_dim * 4, emb_dim)

    def forward(self, x):
        x = self.linear1(x)
        x = nn.GELU()(x) # 非线性激活（增加脑回路的复杂度）
        x = self.linear2(x)
        return x


# ============================================
# 6. 层归一化（拉齐起跑线）
# ============================================
class LayerNorm(nn.Module):
    def __init__(self, emb_dim, eps=1e-5):
        super().__init__()
        self.eps = eps
        # 可学习的参数：缩放(scale) 和 平移(shift)
        self.scale = nn.Parameter(torch.ones(emb_dim))
        self.shift = nn.Parameter(torch.zeros(emb_dim))

    def forward(self, x):
        # 计算平均值
        mean = x.mean(dim=-1, keepdim=True)
        # 计算方差
        var = x.var(dim=-1, keepdim=True, unbiased=False)
```

```
        # 归一化公式：(x - 均值) / 根号(方差 + 极小值)
        norm = (x - mean) / torch.sqrt(var + self.eps)
        return norm * self.scale + self.shift
```




腾讯技术工程

腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。

689篇原创内容

公众号

4. 训练与生成

数据预处理和训练是一件十分庞大和耗时的工程，受限于篇幅，这里只对训练方式和推理做简单介绍，不做展开。

4.1 模型的推理方式

在完成模型结构之后，我们先来尝试一下，看看模型到底会输出什么内容

```
# 首先将输入文本进行编码
# tokenizer.encode()方法将文本字符串转换为模型可理解的数字序列（token IDs）
inputs = tokenizer.encode("QQ浏览器广告后台开发")
print("编码后的token IDs:", inputs)

# 将编码结果转换为PyTorch张量，并添加批次维度
# torch.tensor([inputs])创建了一个形状为[1, sequence_length]的张量
batch = torch.tensor([inputs])

# 将批次数据输入模型进行推理
# model(batch)返回模型的输出，通常是logits或概率分布
output = model(batch)
print("模型输出:", output)
print("输出形状:", output.shape)

# 使用argmax获取每个位置概率最大的token索引
# dim=-1表示在最后一个维度（通常是词汇表维度）上取最大值
predictions = torch.argmax(output, dim=-1)
print("预测结果（token索引）:", predictions)

# 将预测的token索引解码回文本
# tokenizer.decode()将数字序列转换回可读的文本字符串
print("解码后的文本:", tokenizer.decode(predictions[0]))
```

```
```

```
编码后的token IDs: [51, 51, 586, 240, 6262, 1179, 5046, 799, 2507, 3158]
模型输出: tensor([[[ 0.1939, -0.1545, -0.8574,  ...,  0.1755,  0.4264, -0.2223],
         [ 0.4136, -0.0892, -0.4605,  ...,  0.2929,  0.6576,  0.2146],
         [ 0.1227,  0.5149, -0.7861,  ..., -0.2095,  0.1483, -0.0575],
         ...,
         [-0.1255,  0.3100, -0.4338,  ..., -0.4569,  0.1680, -0.2783],
         [ 0.0123, -0.2317,  0.1034,  ..., -0.6657,  0.4466, -0.1969],
         [-0.0439, -0.2680, -0.8711,  ..., -0.8857,  0.7761,  0.2247]]],
       grad_fn=<ViewBackward0>)
输出形状: torch.Size([1, 10, 6400])
预测结果（token索引）: tensor([[1536, 1536,  738, 5155, 4507, 5155, 2857, 4672, 3506, 3283]]
解码后的文本: 什什 R安排 similar安排软件 diagnosonom
```

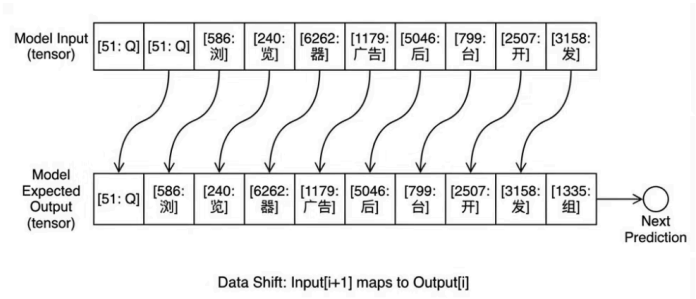
可以看到，输入 9 个 Token 后，模型对应输出了 9 个向量，每个向量的长度都等于词表大小。这些数值可以理解为模型对词表中每个词的‘打分’(或者说概率，这里省略了softmax处理)。如果我们选取每个向量中数值最大的那个（即模型认为概率最高的词），并将其还原为文字，就得到了模型最终的预测结果。

目前模型返回的数据还是毫无逻辑的token，因为我们还没进行训练，接下来我们就让他变聪明些。

4.2 训练

4.2.1 数据构造

我们先要知道，我们期望模型返回的数据格式



对于输入的 "QQ浏览器广告后台开发", 我们期望模型整体前推一个字，即输出 "Q浏览器广告后台开发组"，这里的"Q浏览器广告后台开发"，与我们的input相同，我们希望通过这种方式让模型学习到语言模式，而第n个字"组"即是我们想要的字。

```
# 原始语料数据（假设这是分词后的 ID 序列）
# 对应文本: Q   Q   浏   览   器   广   告   后   台   开   发   组
data = [51, 51, 586, 240, 6262, 1179, 5046, 799, 2507, 3158, 1335]
data_tensor = torch.tensor(data)
# 1. 构造输入 (Input): 取序列的前 N-1 个字
# 也就是: "QQ浏览器广告后台开发"
x = data_tensor[:-1]
```

```
# x = [51, 51, 586, ..., 3158]
# 2. 构造目标 (Target): 取序列的后 N-1 个字 (即每个位置对应的"下一个字")
# 也就是: "Q浏览器广告后台开发组"
y = data_tensor[1:]
# y = [51, 586, 240, ..., 1335]
print(f"输入长度: {len(x)}, 目标长度: {len(y)}")
# 输入长度: 11, 目标长度: 11 (一一对应)
# 3. 扔进模型计算
logits = model(x.unsqueeze(0)) # 增加 batch 维度
# 4. 计算 Loss (交叉熵损失)
# 这一步是在问模型:
# 当输入第1个'Q'时, 你预测是第2个'Q'的概率大吗?
# 当输入'浏'时, 你预测是'览'的概率大吗?
# ...
# 当输入'发'时, 你预测是'组'的概率大吗?
loss = nn.cross_entropy(logits.view(-1, vocab_size), y.view(-1))
loss.backward() # 反向传播, 修正参数
```

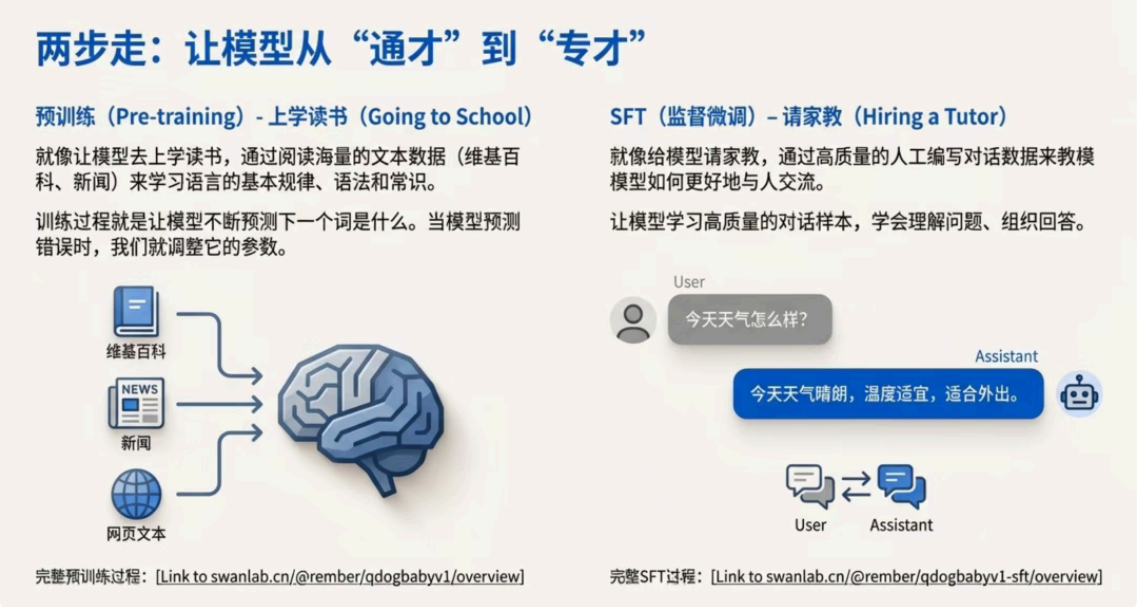
这里有个反直觉的点：虽然我们在逻辑上是“预测下一个字”，但在训练时，模型是并行看到所有输入的。它不需要等“浏”字生成了再算“览”字，而是一次性计算所有位置的预测误差(由于3.4节 causal mask 的存在，模型不会发生"看答案"的情况)。

通过这种方式，模型学会了：

- 看到 "QQ浏览"，大概率要接 "器"
- 看到 "后台开"，大概率要接 "发"
- 看到 "开发"，大概率要接 "组" (或者 "者"、"工程师")

这就是大模型“学会说话”的根本原因。

4.2.2 预训练 与 SFT



这里补充说明一下SFT时的数据构造情况，我们通常会定义一些特殊的 Token（如<|im_start|>）来告诉模型哪里是人说的，哪里是它该说的，因此我们构造的对话数据一般为这种格式:

```
# SFT 数据构造示例
sft_template = """
<|im_start|>user
请介绍一下QQ浏览器。<|im_end|>
<|im_start|>assistant
QQ浏览器是腾讯开发的一款...<|im_end|>
"""
```

在 SFT 阶段，我们只计算 assistant 回答部分的 Loss。这就好比考试评分：老师只给你的“答案”打分，而不会因为你把“题目”抄写得好不好而扣分。 这样做的目的是让模型专注于学习“如何根据问题给出正确回答”，而不是去死记硬背问题本身。

4.3 采样策略

在模型生成文本时，我们通常不希望每次都选择概率最高的词，这样会让输出变得过于确定和单调。温度和topk就是用来控制生成多样性的两个重要参数。

温度（Temperature） 就像是给模型的"创造力"调个温：

- 低温（如0.1）：模型更保守，总是选择最可能的词，输出稳定但可能单调
- 高温（如0.5-1.0）：模型更"放飞自我"，会尝试更多可能性，输出更有创意但可能不够准确

Top-k采样 则是另一种控制方式：

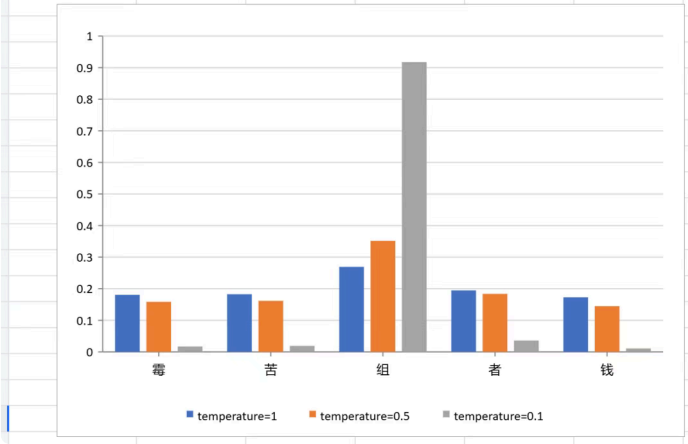
- 只从概率最高的k个候选词中选择，避免选择那些概率极低的词
- 比如k=4，就只从前4个最可能的词中随机选择

这两种方法可以单独使用，也可以结合使用，让模型在"靠谱"和"有趣"之间找到平衡。

```
# QQ浏览器广告后台开发 + ？？？
# vocab: [霉, 苦, 组, 者, 钱]
probas = torch.tensor([0.1145, 0.1245, 0.5130, 0.1887, 0.0694])
print(torch.softmax(probas, dim=-1))
temperature = 0.5
print(torch.softmax(probas/temperature, dim=-1))
temperature = 0.1
print(torch.softmax(probas/temperature, dim=-1))
print(torch.topk(probas, k=4, dim=-1))
```

```
tensor([0.1807, 0.1826, 0.2692, 0.1947, 0.1728])
tensor([0.1584, 0.1616, 0.3515, 0.1837, 0.1447])
tensor([0.0171, 0.0189, 0.9174, 0.0358, 0.0109])
torch.return_types.topk(
values=tensor([0.5130, 0.1887, 0.1245, 0.1145]),
indices=tensor([2, 3, 1, 0]))
```

token	霉	苦	组	者	钱
temperature=1	0.1807	0.1826	0.2692	0.1947	0.1728
temperature=0.5	0.1584	0.1616	0.3515	0.1837	0.1447
temperature=0.1	0.0171	0.0189	0.9174	0.0358	0.0109



通过上面的例子我们发现 Temperature 系数实际上是在调整 Softmax 之前的 Logits 数值分布。 T 减小 (冷却)：拉大 Logits 之间的差距，高的更高，矮的更矮。概率分布变得“尖锐”，模型只敢选最有把握的词。 T 增大 (加热)：缩小 Logits 之间的差距，众生平等。概率分布变得“平坦”，原本概率低的词也有机会被选中，模型开始“胡言乱语”或“创意迸发”。

5. 演示

预训练后的模型:



这是因为预训练模型本质上是一个**文本续写机**。它在海量文档中学习的是“下一个字是什么”的概率分布。在它眼里，你输入的问题可能只是某本小说或新闻的开头，它并不知道这代表一次

“对话”的开始。它只是在尽职尽责地根据概率，顺着你的话继续往下编故事，而不是在回答你的问题。

SFT后的模型:



通过SFT，我们让模型学会了对话的模式，模型真正的完整回答问题。

TIPS: 由于网络环境/资源等原因，演示模型没有对外部署，感兴趣的同学可以使用 **minimind** 提供的数据集，配合 **codebuddy** 快速生成训练逻辑，训练出一个属于自己的LLM。

6. 参考资料

1. 《Build a Large Language Model (From Scratch)》
2. CS336: Language Modeling from Scratch
3. The Illustrated Transformer
4. The Illustrated GPT-2 (Visualizing Transformer Language Models)
5. The Big LLM Architecture Comparison
6. minimind
7. Visualizing A Neural Machine Translation Model (Mechanics of Seq2seq Models With Attention)
8. 《Hands-On Large Language Models》
9. Happy-LLM

中文数据集:

1. MiniMind训练数据集

2. 出门问问序列猴子开源数据集

3. opencsg/chinese-fineweb-edu-v2

4. 匠数科技大模型sft数据集



腾讯技术工程
腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。
689篇原创内容
公众号



阅读 5.6万

留言 31

写留言

🥕🐰 浙江 2025年12月24日 110

“看到这里的你已经成功手搓第一个模型了，下一个任务：干掉Gemini3pro🤓🖕”

毅璐向前 山东 2025年12月24日 33

进入我的收藏吃灰吧

明安 广东 2025年12月24日 17

人家的LLM是Large Language Modle，接下来我们手搓一个Little Language Modle吧😋

花开富贵 湖北 2025年12月24日 16

竟然试图教会我大模型🤭

安曲 新疆 2025年12月24日 11

我现在备注大模型算法岗就在自己手搓大模型，技术路线居然和这篇文章不谋而合。昨天已经手搓单头注意力机制了，明天手搓多头注意力机制。

北斗之下 浙江 2025年12月24日 11

宝藏资料！引用的都是最近在看的！再推荐一个就是karpethy github项目的nanoGPT

@小·边·边 内蒙古 2025年12月26日 9

《如何0基础手搓原子弹》😹

Xinn 北京 2025年12月24日 6

看完了，没学费，能不能来一个手把手教学🤫

格心 浙江 2025年12月24日 4

这使用了多少显存？硬件要求是怎么样的

爱做梦的大… 湖南 2025年12月24日 3

多写，爱看

李鸿儒 天津 2025年12月24日 2

你已经会本地训练小模型了，现在去训练一个340B的大模型吧😹

田禾子 上海 2025年12月24日 2

写的很棒呀！逻辑清晰步骤明确 www，这段时间看的讲的最清楚的一篇手搓 llm 了

StackRising 湖南 2025年12月24日 1

这本书确实值得一读，写的很详细

CIA·兵 广东 2025年12月24日 1

好棒啊

千的乐 海南 2025年12月24日 1

👍

Reprise 浙江 2025年12月24日 1

可以，

星河 北京 2025年12月24日 1

感动了，真的太感动了，写得特别好

。 湖北 2025年12月24日 1

写的真好，可惜我看不懂🙄

Kerwin的地盘 天津 2025年12月24日 1

里边的图用什么画的

GORIT 湖北 2025年12月24日 1

@元宝 总结一下

xz 江苏 2025年12月24日 1

这个好啊！还有后续的教程吗

cutemurphy 浙江 2025年12月24日 1

AI会帮我写

bear 四川 2025年12月26日

从第二节开始就略过了

刘荣 山东 2025年12月26日

头大，有点难

8 月失败的 … 我 广东 2025年12月24日

@元宝 总结文章，怎么做一个元宝出来

DK坚果壳 北京 2025年12月24日

喜欢这种，有时间自己写写试试

Good. 广东 2025年12月24日

点赞在看=学会了

yo. 广东 2025年12月24日

马上动手！

FireFliesə🐷 ɢ 上海 2025年12月24日

收藏，晚点再看

Jenson 北京 2025年12月24日

今天就先看到1.4节，累了🙄

付@软件开发 四川 2025年12月24日

手搓一个操作系统，一个编辑器，一个大模型。。😁